

Doubly Linked List – Delete From End And Time Complexity.

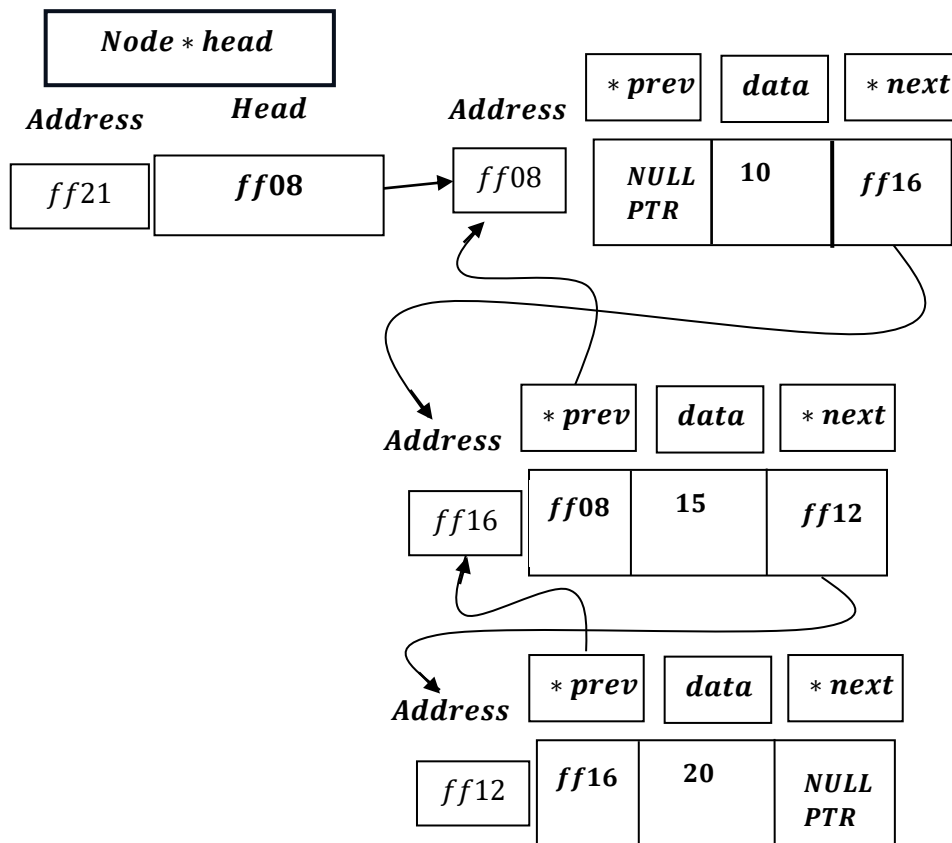
Program :

```
void deleteFromEnd()
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }
    Node *temp = head;
    if (temp->next == nullptr)
    { // Only one node in the list
        head = nullptr;
    }
    else
    {
        while (temp->next != nullptr)
        {
            temp = temp->next; // Traverse to the last node
        }
        temp->prev->next = nullptr;
        temp->prev = nullptr;

        cout << "Deleted " << temp->data << " from the end." <<
endl;
        free(temp);
        temp = nullptr;
    }
    ... .
case 6:
    deleteFromEnd();
    break;
```

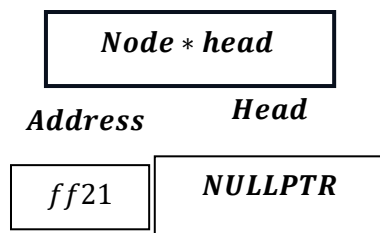
Program Analysis:

Lets consider a Doubly Linked List :



Now, first Line of Code:

```
if (head == nullptr)
{
    cout << "List is empty. Nothing to delete." << endl;
    return;
}
```

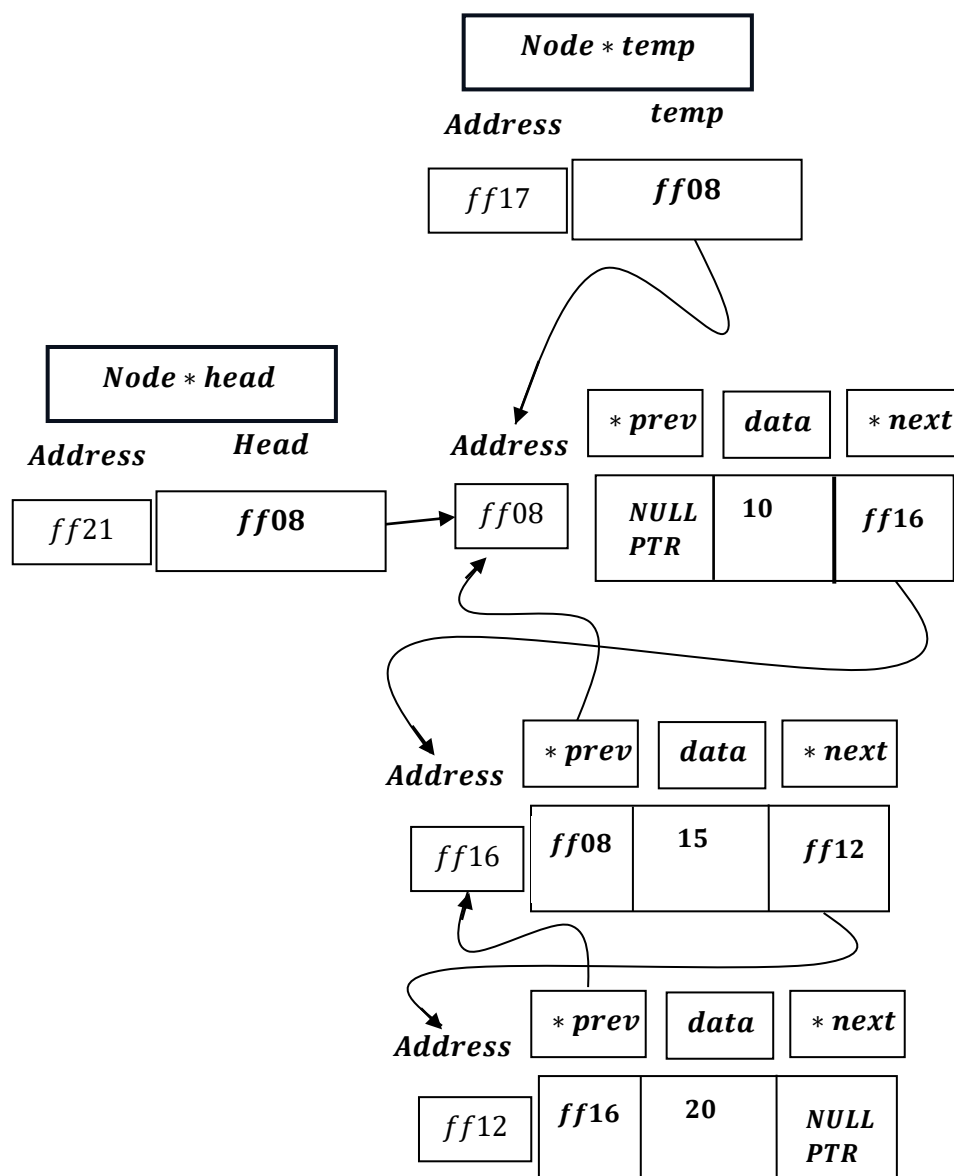


Then, it will print statement : `` List is empty. Nothing to delete.``

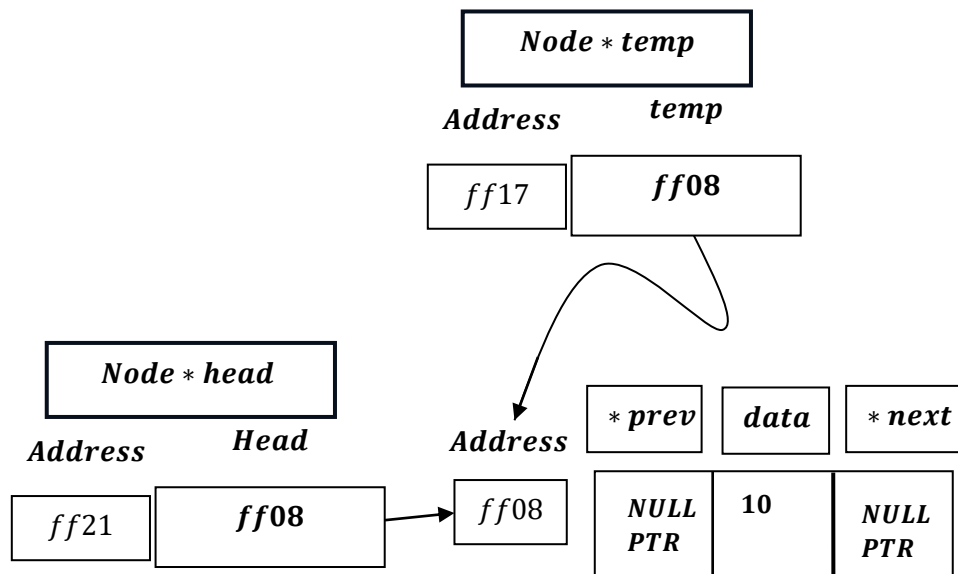
And, return statement will exit from the function.

Next, Line of Code:

Node
*temp
=
head
;

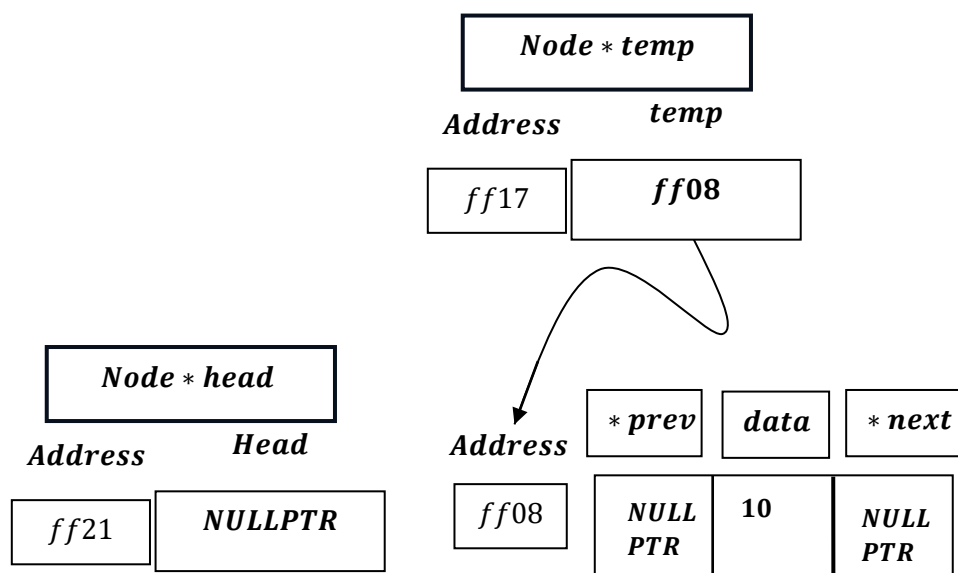


1. If there is single node :



```
if (temp->next == nullptr)
{
    head = nullptr;
}
```

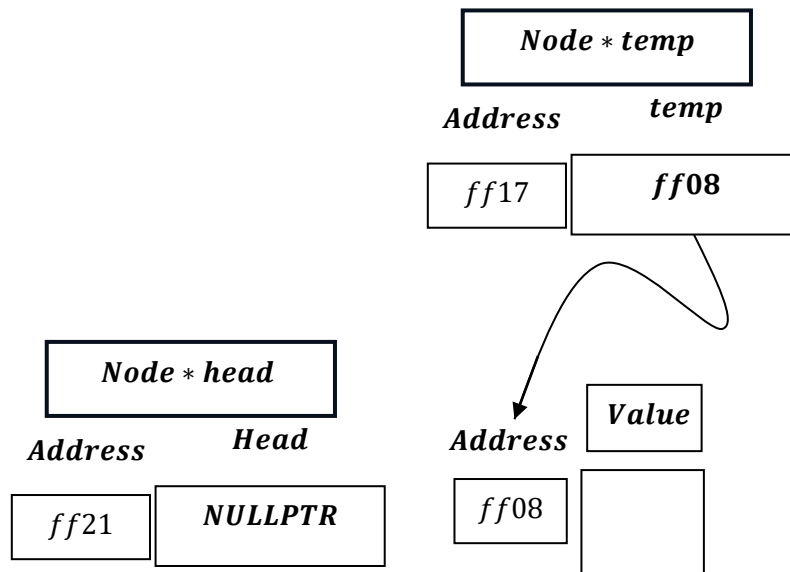
The condition: $temp \rightarrow next = nullptr$ is true , then if block's statement gets executed.



Next,

```
free(temp);
```

This will free the memory location that temp pointed to:

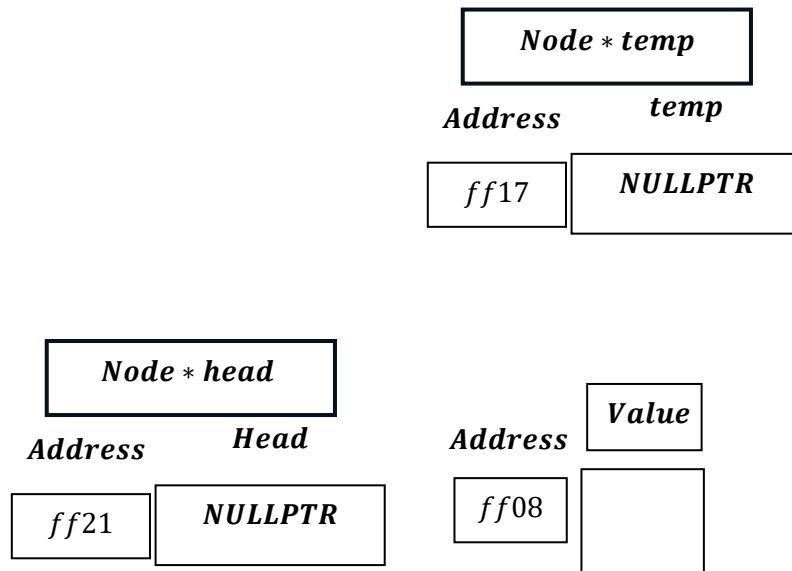


Now, temp is a dangling pointer. Now, the memory location after freeing , is not a valid memory location , hence the pointer `temp` is now a dangling pointer.

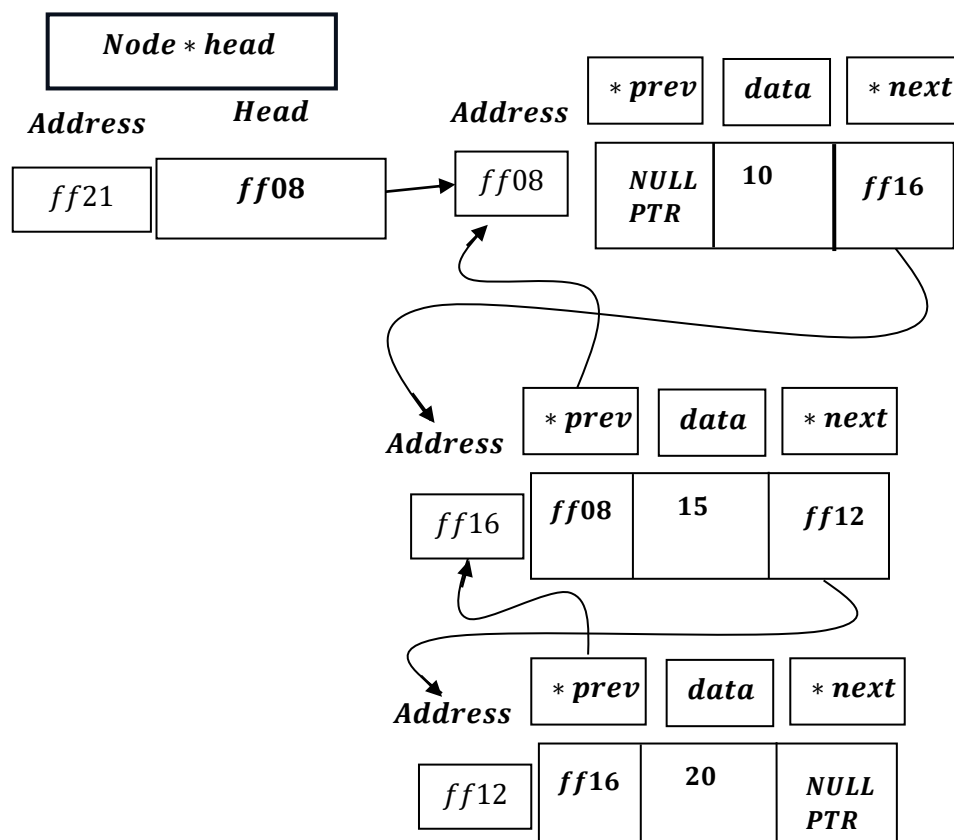
A dangling pointer is a pointer that continues to exist even after the memory it points to has been deallocated or freed. It's called dangling because it is no longer points to a valid object.

Hence,

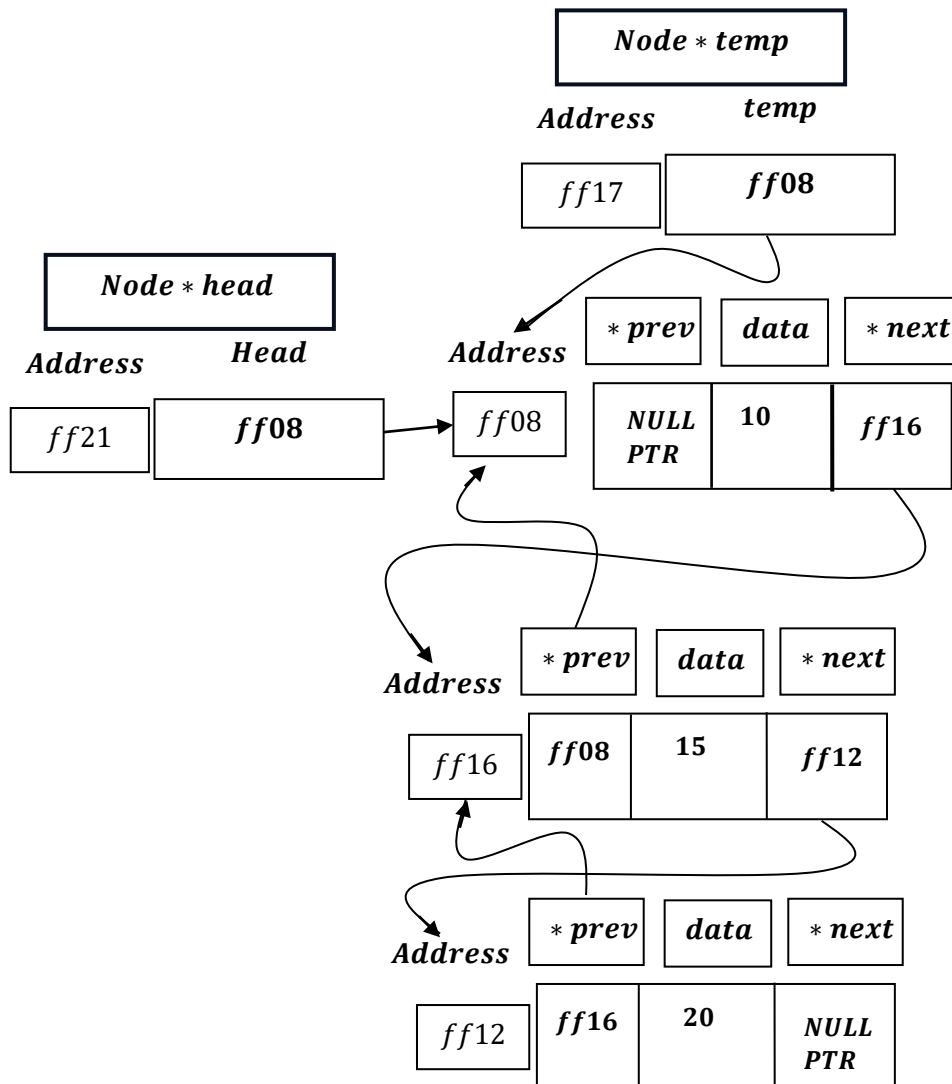
```
temp = nullptr;
```



2. If there is 'n' no. of nodes:

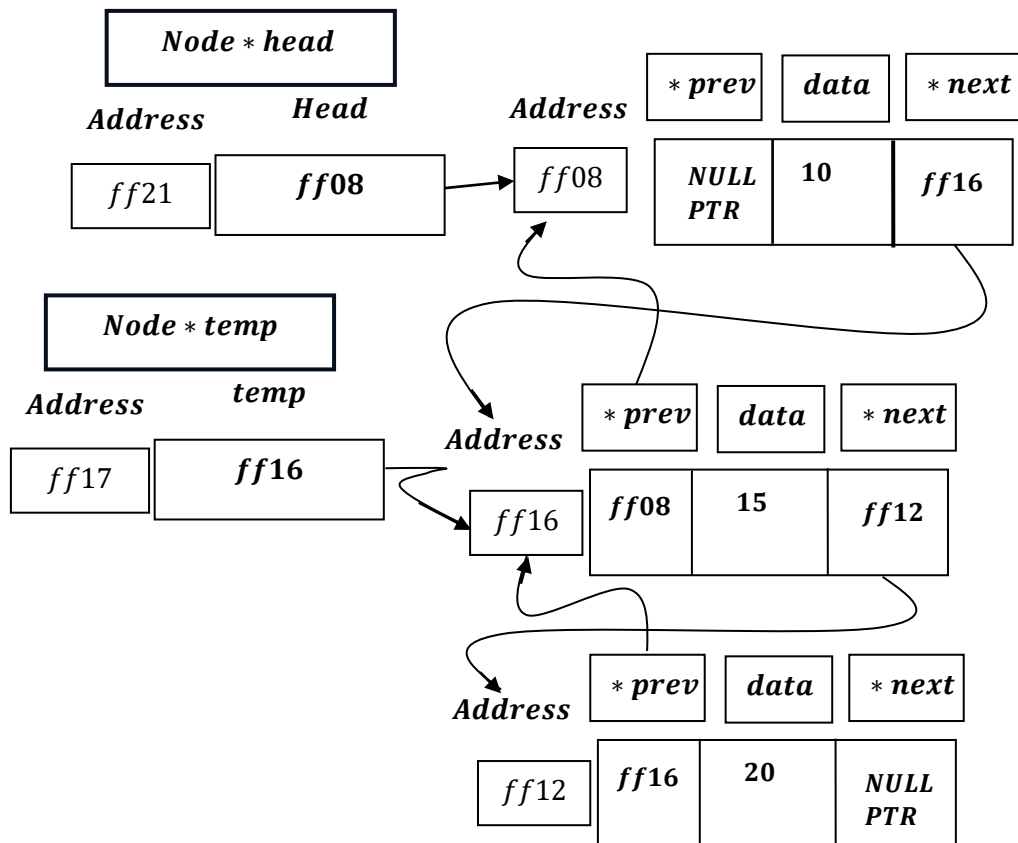


```
Node *temp = head;
```

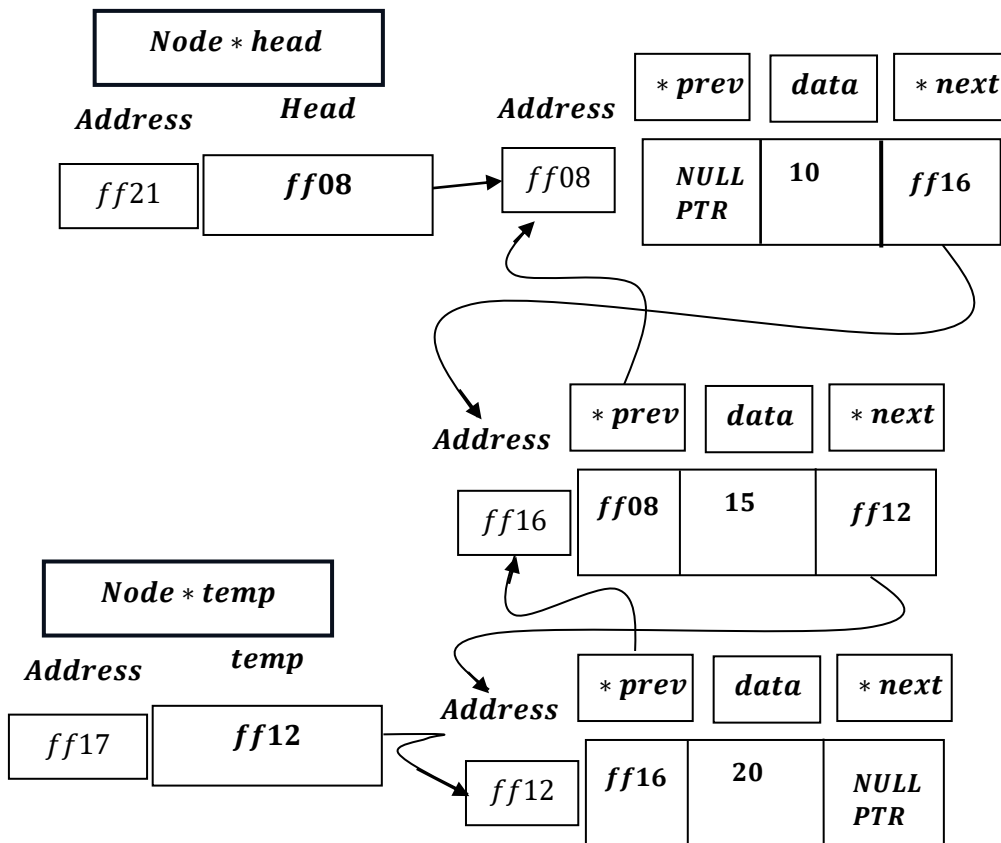


```
if (temp->next == nullptr){... }
else
{
    while (temp->next != nullptr)
    {
        temp = temp->next; // Traverse to the last node
    }
    .....
}
```

temp → *next* ≠ *nullptr* is true , then: *temp* = *temp* → *next* = ff16.



temp → *next* ≠ *nullptr* is true , then: *temp* = *temp* → *next* = *ff12*.



temp → *next* ≠ *nullptr* is false , hence loop condition becomes false , and loop exits.

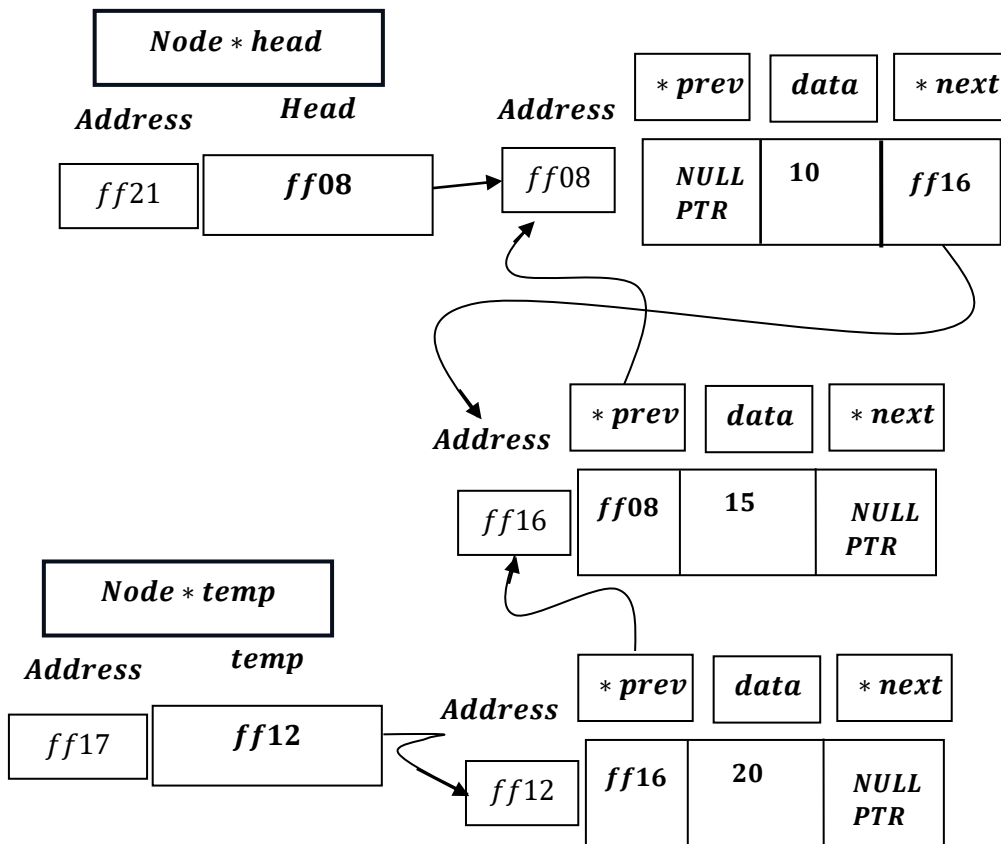
```

if (temp->next == nullptr){... }
else
{
    while (temp->next != nullptr){ .....}
    temp->prev->next = nullptr;
    .....
}

```

i.e.,

```
temp->prev->next = nullptr;
```



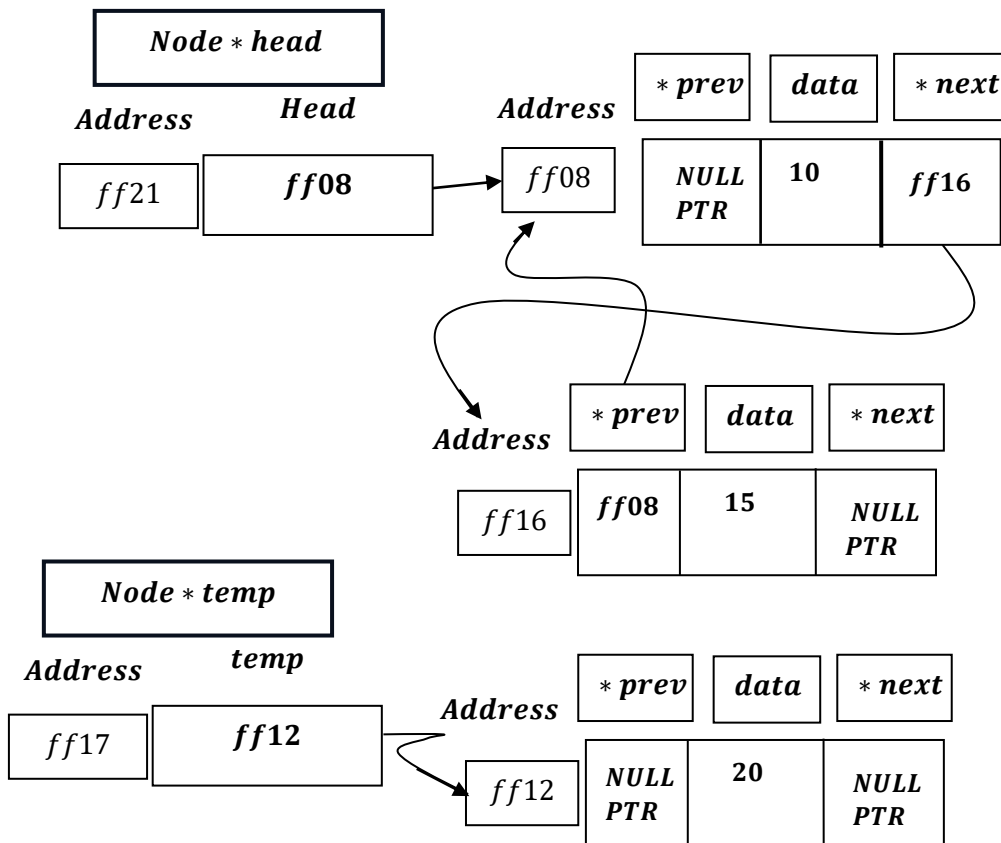
```

if (temp->next == nullptr){... }
else
{
    while (temp->next != nullptr){ .....}
    temp->prev->next = nullptr;
    temp->prev = nullptr;
}

```

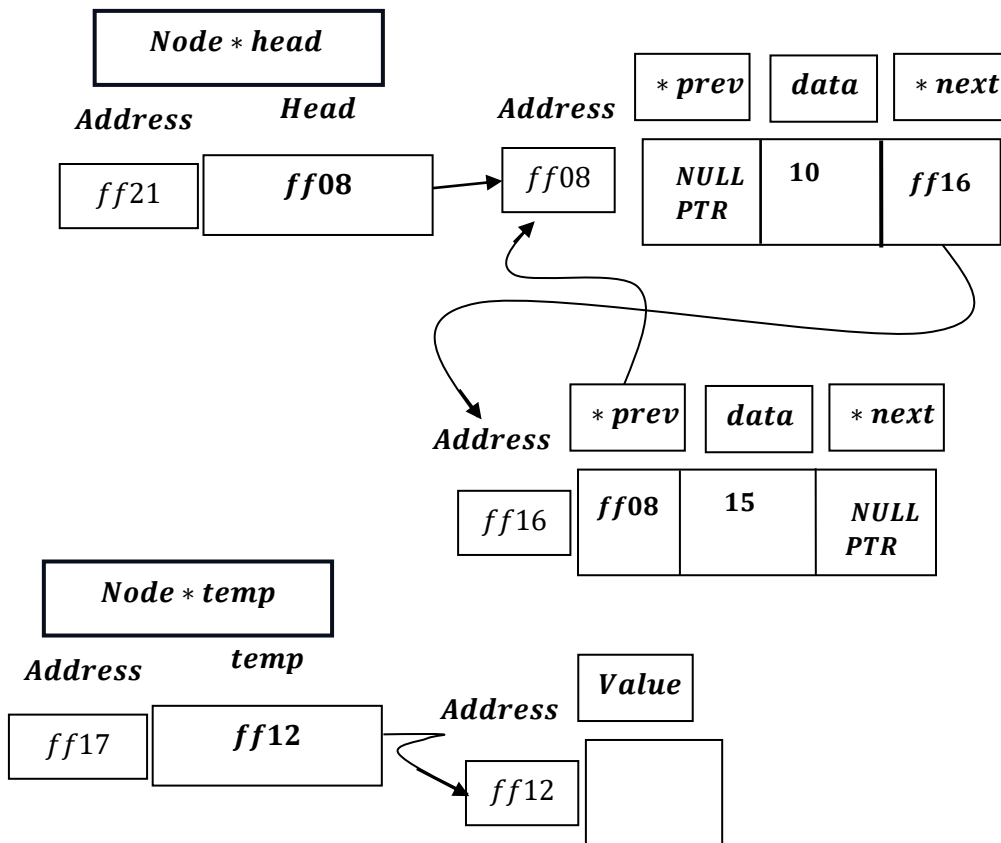
i.e.,

```
temp->prev = nullptr;
```



Next,

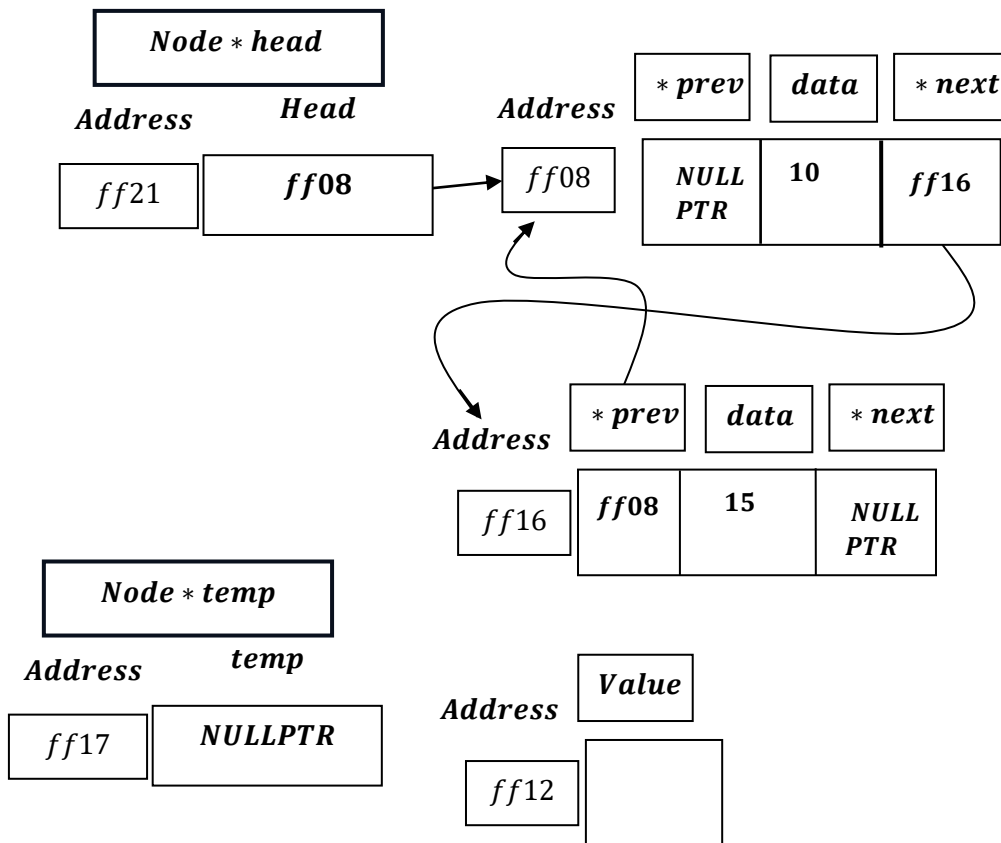
```
free(temp);
```



Now, temp is a dangling pointer. Now, the memory location after freeing, is not a valid memory location, hence the pointer `temp` is now a dangling pointer.

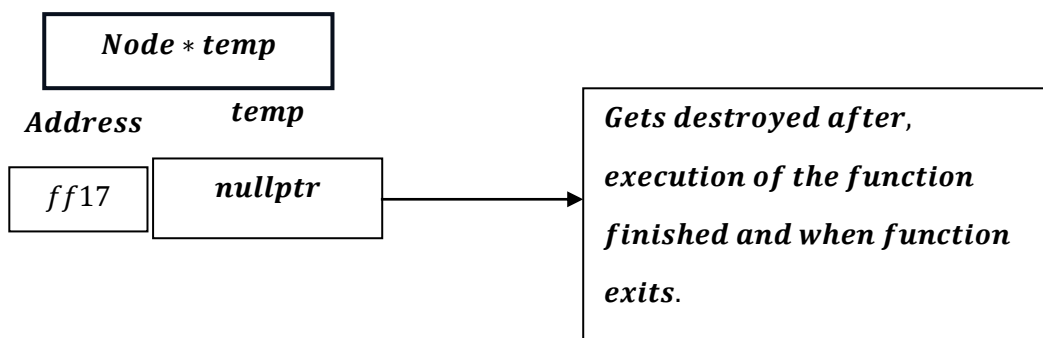
A dangling pointer is a pointer that continues to exist even after the memory it points to has been deallocated or freed. It's called dangling because it is no longer points to a valid object.

```
temp = nullptr;
```

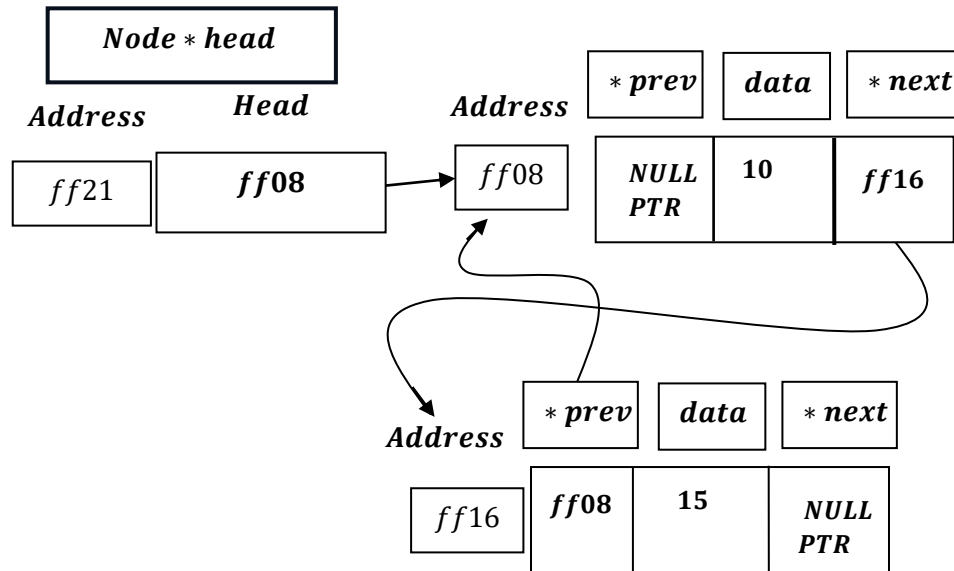


temp = nullptr; → This ensures that any future checks on the pointer (e.g., if (temp != NULLPTR)) will correctly prevent the program from trying to use the invalid memory location.

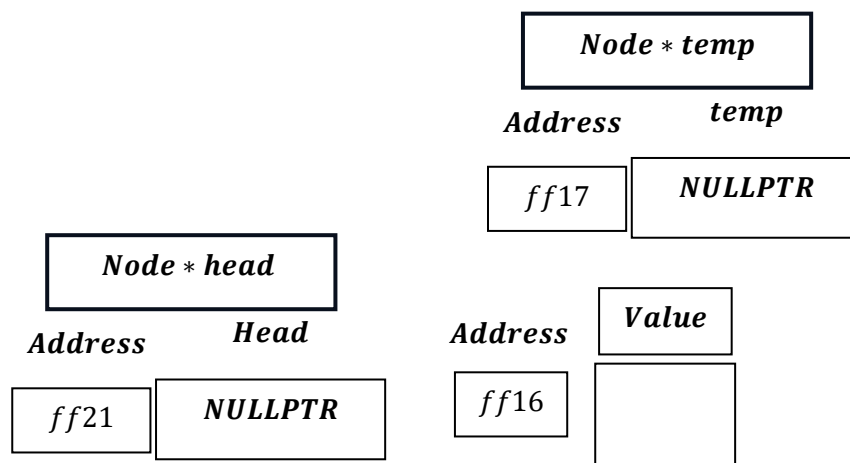
*Node * temp temporary variable, therefore gets destroyed after function execution get finished.*



Hence ,we are left with:



Note: –After ,deleting from beginning,List can become Empty.



Then again :

```
if (head == nullptr)
{
    cout << "List is empty. Nothing to delete." << endl;
    return;
}
```

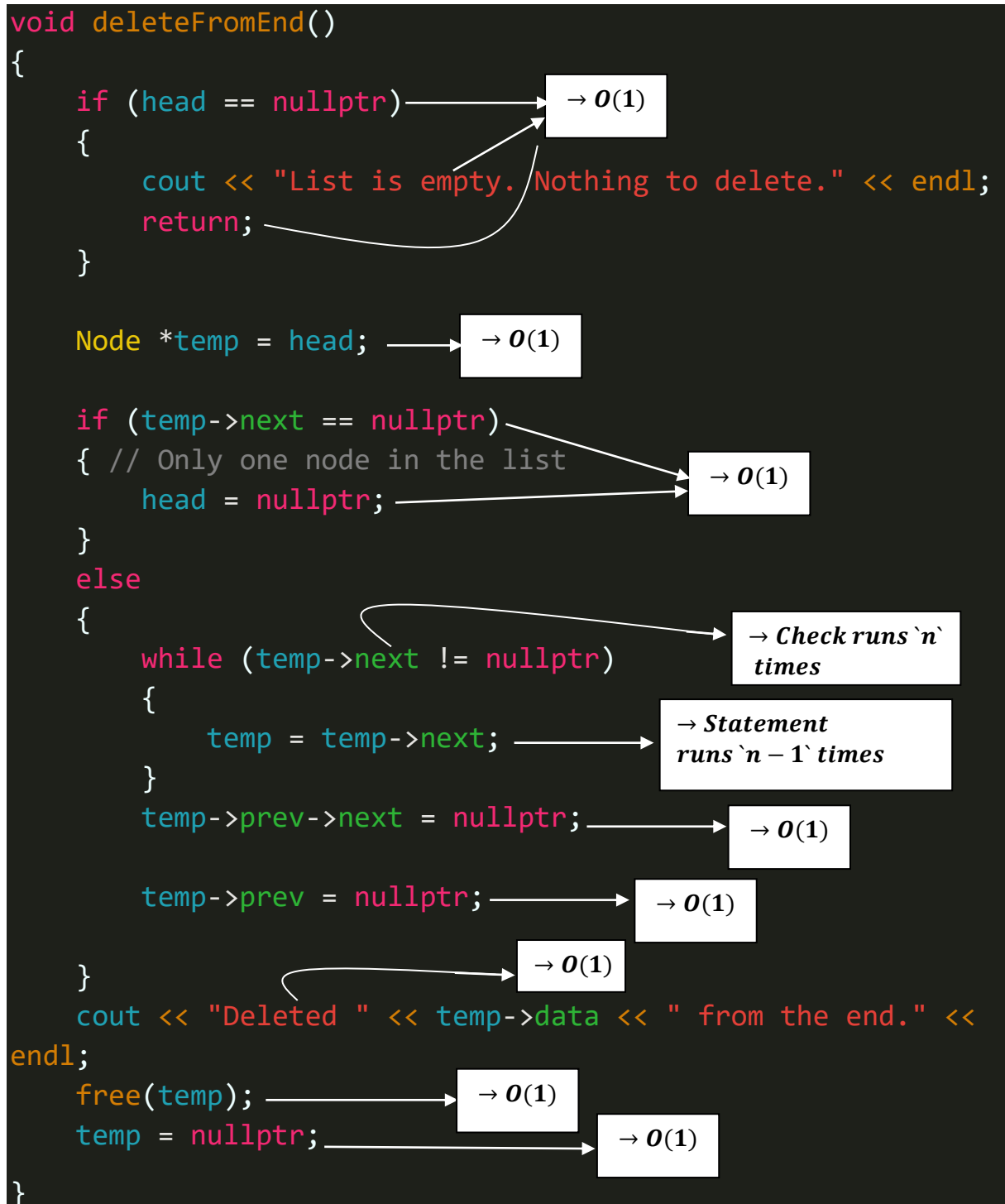
will run and return statement will make exit from the function.

Doubly Linked List – Delete From End [Working Summary]:

This function removes the last node from a doubly linked list.

1. **Handles Empty List:** It first checks if the list is empty (`head == nullptr`). If it is, the function prints a message and exits.
 2. **Handles Single-Node List:** It checks if the first node is also the last node (`temp->next == nullptr`). If so, it sets the `head` to `nullptr`, effectively emptying the list.
 3. **Finds the Last Node:** If the list has multiple nodes, it uses a `while` loop to traverse the list until `temp` points to the very last node (`temp->next` is `nullptr`).
 4. **Detaches the Last Node:**
 - `temp->prev->next = nullptr;` This is the key step. It accesses the second-to-last node (`temp->prev`) and sets its `next` pointer to `nullptr`, officially making it the new end of the list.
 - `temp->prev = nullptr;` The `prev` pointer of the node to be deleted is also nullified.
 5. **Deallocates Memory:** Finally, it frees the memory that was used by the last node (pointed to by `temp`) and then sets the `temp` pointer itself to `nullptr` to prevent it from becoming a dangling pointer.
-

Time Complexity of Delete From End Function Module



1. Best Case ,when there is single head node ,where head pointer pointing to the single head node or List is Empty.

1. When List is Empty[Absolute Best Case]:

→ If `head = nullptr`, this checking takes constant time. The `cout` and `return` statements take constant time. Hence overall process takes, $O(1)$ time.

2. When List has a single Node[Best Case but not Absolute]:

→ If `head = nullptr` which is false, this checking takes constant time. Hence overall process takes, $O(1)$ time.

→ Temp pointer is assigned to Head Node, which makes Temp pointer to point at memory location to which Head pointer is pointing to, this process takes constant amount of time i.e. $O(1)$.

→ If condition checks next pointer in memory location pointed by temp pointer of Node named structure whether equals to `NULLPTR`, if equals to `NULLPTR`, then, it is approved that List has a single node, thereafter, if block statement runs i.e. Head pointer gets assigned to `NULLPTR` making LIST empty. this process takes constant amount of time i.e. $O(1)$.

→ Next, it prints out, data which is to be deleted, this process takes constant amount of time i.e. $O(1)$.

→ `free(temp)` makes allocated memory pointed by temp reusable, this takes constant amount of time i.e. $O(1)$.

→ Then to avoid dangling pointers, temp pointer of Node named structure set to `nullptr`, this takes constant amount of time i.e. $O(1)$.

Total Time Complexity: $O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(1)$

Why not Absolute Best Case?

**Ans: – Since here for single node, it needs more `Line of Codes` to execute ,
than for Empty List , therefore more operation to be executed , hence ,
For single node, its not the absolute best case.**

2. Worst Case , when there present multiple nodes in the linked list.

→ If head = nullptr which is false , this checking takes constant time.

Hence overall process takes, $O(1)$ time.

**→ Temp pointer is assigned to Head Node , which makes Temp pointer to point at
memory location to which Head pointer pointing to , this process takes constant
amount of time i. e. $O(1)$.**

**→ If condition checks next pointer in memory location pointed by temp pointer
of Node named structure whether equals to NULLPTR, the check is false , hence,
this process takes constant amount of time i. e. $O(1)$.**

→ As if block is false , hence else block gets executed : –

**→ So, while condition checks $temp \rightarrow next \neq NULLPTR$
check runs `n` times as last check will be false hence ,
inner statement i. e. traversal: $temp = temp \rightarrow next$
gets executed `n – 1` time , we consider loop statement
execution here , hence $O(n - 1) = O(n)$.**

→ Now Temp pointer pointed to Memory Allocation's previous pointer pointed to previous Memory Allocation or previous node's Next pointer sets to NULLPTR, this detach Next pointer of Previous node pointed to allocated memory pointed by Temp pointer. This process takes constant amount of time i.e. $O(1)$.

→ Now we make Temp pointer pointed to Memory Allocation's previous pointer set to NULL, which again again detaches previous pointer from pointing to previous Node. This process takes constant amount of time i.e. $O(1)$.

→ Next, it prints out, data which is to be deleted, this process takes constant amount of time i.e. $O(1)$.

→ `free(temp)` makes allocated memory pointed by temp reusable, this takes constant amount of time i.e. $O(1)$.

→ Then to avoid dangling pointers, temp pointer of Node named structure set to 'nullptr', this takes constant amount of time i.e. $O(1)$.

Total Time Complexity: $O(1) + O(1) + (O(1) + (O(n) + O(1) + O(1))) + O(1) + O(1) = O(n)$.

We can say, if List is Empty, or we have a single head node connected to Linked List, we get our best case scenario, and when list is not empty and there present multiple nodes, we get our worst case scenario. But though best case and worst case scenarios are present as, always we can determine where the deletion to be made, hence we can say average case is our worst case here i.e. always the entire list is traversed i.e. $O(n) = \Theta(n)$.

In the case of DeleteAtEnd, the deletion always happens at the end, regardless of the current state of the list. This means the execution time is solely dependent on the length of the list. If the list is empty, the execution time is constant (best case). If the list is not empty, the execution time is proportional to the length of the list (worst case). Since the deletion position is predetermined, there's no element of randomness or varying input data to consider. Therefore, the concept of an average case, where we average the execution time across different input possibilities, doesn't strictly apply here. Hence DeleteAtEnd doesn't have a true average case.

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq N \leq c_2 \times N$$

<i>Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>DeleteAt End</i>	<i>1. Best Case</i>	$\Omega(1)$
	<i>2. Worst Case</i>	$O(n)$
	<i>3. Average Case</i>	$\Theta(n)$
